/spec

Specification management: /spec create|add|delete|list|search|validate|status|task|help

Overview

`/spec` manages the full lifecycle of specification documents stored under the project's `specs/` directory. Each spec is a folder containing a `SPEC.md` (requirements and tasks), optional `PLAN.md` and `TESTPLAN.md` artifacts, and status metadata that `/spec` tracks through draft, review, approved, implemented, verified, and archived states. The command family covers creation, searching, validation, task tracking, JTBD analysis, and lifecycle transitions. Every form reports results in the message window as a table or status block.

Syntax

```
/spec                          # list active specs (default when no args)
/spec help                     # show the subcommand table
/spec create <name>            # scaffold a new spec folder
/spec add <name>               # add a spec discovered on disk into the registry
/spec delete <name>            # delete a spec
/spec validate <name>          # validate spec structure and requirements
/spec list [--status <s>] [--prefix <p>]
/spec search <query>           # keyword search across spec titles and content
/spec status <name> [<new-status>]
/spec task <name>              # task operations for a spec
/spec activate <name>          # mark a spec as the active spec
/spec deactivate               # clear the active spec
/spec coverage <name>          # requirement-to-task coverage report
/spec impl|implement <name> [--task <ID>] [--dry-run]
/spec jtbd <name> [--force] [--agent <name>]
/spec update <name>            # regenerate PLAN.md / TESTPLAN.md from SPEC.md
/spec specify <name>           # edit or regenerate the SPEC.md body
/spec plan <name>              # regenerate PLAN.md
/spec tasks <name>             # regenerate TESTPLAN.md
/spec feedback <name>          # manage review feedback (REVIEW.md / FEEDBACK.md)
```

Subcommands

Form	Description
`/spec` (bare)	Lists specs in the current status view.
`/spec help`	Prints the subcommand table.
`/spec create <name>`	Creates a new spec folder with a starter SPEC.md.
`/spec add <name>`	Registers an existing spec folder found on disk.
`/spec delete <name>`	Removes the spec from the registry and disk.
`/spec validate <name>`	Checks structure, requirement IDs, and task links.
`/spec list [--status <s>] [--prefix <p>]`	Lists specs, filtered by status or name prefix.
`/spec search <query>`	Keyword search across titles and content.

Form	Description
`/spec status <name>`	Shows the spec's lifecycle status.
`/spec status <name> <s>`	Transitions the status (draft, in_review, approved, in_progress, implemented, verified, archived).
`/spec task <name>`	Task-tracking operations for the spec's plan.
`/spec activate <name>`	Sets the active spec used by /spec update, /spec tasks, and spec task tools.
`/spec deactivate`	Clears the active spec.
`/spec coverage <name>`	Renders the `[ok]`/`[wait]`/`[sync]`/`[stop]` coverage report linking requirements to completed tasks.
`/spec impl <name>`	Marks the spec as implementing; creates-or-updates session tracker tasks seeded from PLAN.md. `--task <ID>` targets one task, `--dry-run` previews without writing.
`/spec implement <name>`	Alias of `/spec impl`, same flags.
`/spec jtbd <name>`	Performs a Jobs-To-Be-Done analysis on the spec. `--force` overwrites prior analysis; `--agent <name>` runs the analysis through a specific agent.
`/spec update <name>`	Regenerates PLAN.md and TESTPLAN.md from an edited SPEC.md.
`/spec specify <name>`	Regenerates or edits the SPEC.md content itself.
`/spec plan <name>`	Regenerates PLAN.md only.
`/spec tasks <name>`	Regenerates TESTPLAN.md only.
`/spec feedback <name>`	Manages the spec's feedback loop (REVIEW.md / FEEDBACK.md).

Examples

`/spec create auth-refactor`

Creates the `specs/auth-refactor/` folder with a starter SPEC.md.

`/spec list --status in_review --prefix auth`

Lists in-review specs whose names start with `auth`.

`/spec coverage auth-refactor`

Shows which requirements are linked to completed tasks and which are pending.

`/spec implement auth-refactor --dry-run`

Previews the implementation task update without touching tracker state.

`/spec jtbd auth-refactor --agent general`

Runs a Jobs-To-Be-Done analysis on the spec through the general agent.

`/spec update auth-refactor`

Regenerates PLAN.md and TESTPLAN.md after SPEC.md edits.

Output

- Lists render as spec tables with name, status, and metadata columns.
- Coverage reports use `[ok]` (requirement covered by completed tasks), `[wait]` (tasks pending), `[sync]` (drift between spec and tracker), and `[stop]` (blocked) task-status symbols.
- Lifecycle transitions print a confirmation with the old and new status.
- Validation failures list each problem with a pointer to the offending requirement or task ID.
- Spec file writes are atomic (temp file plus rename). Writing an empty REVIEW.md or FEEDBACK.md clears it. Task-completion heuristics only fire for writes inside the active spec directory.

Related

- `/spec coverage` semantics are shared with the spec_task_update tool.
- `ragent-specs` crate implements the lifecycle engine behind every form.
- specs//SPEC.md is the source of truth; PLAN.md and TESTPLAN.md are derived artifacts.
- See docs/howtos/ for the spec system manual and JTBD analysis guide.